# E-Token Backend — C# / .NET Architecture Guide

Prepared by: Mohammed | Mobile App Development | Jaiz Bank

## 1. Target stack

| Layer | Choice | Notes |
|---|---|---|
| Runtime | **.NET 10 (LTS)** | Current LTS, released Nov 2025, supported through Nov 2028 — right choice for new bank infrastructure over the STS .NET 9 line |
| Web framework | **ASP.NET Core Web API** | Minimal APIs or controller-based, either is fine; controllers read more familiar for a team coming from NestJS |
| ORM | **Entity Framework Core 10** | Matches runtime version |
| TOTP library | **Otp.NET** | Implements RFC 6238 directly — do not hand-roll the HMAC/truncation logic |
| Secrets encryption | **Azure Key Vault** or **AWS KMS** (whichever the bank's infra uses), accessed via the corresponding .NET SDK | Application code never handles raw key material directly — always through the vault's encrypt/decrypt calls |
| Database | SQL Server or PostgreSQL (match existing core banking infra) | Schema in section 4 is engine-agnostic |
| Rate limiting | **Microsoft.AspNetCore.RateLimiting** (built into ASP.NET Core) | Native middleware, no extra dependency needed |

## 2. Solution structure (Clean Architecture)

The diagram above shows the shape; here's the concrete project layout:

```
EToken.sln
├── EToken.Api/                  → Controllers, DTOs, middleware, Program.cs
├── EToken.Application/          → Use cases: EnrolCustomer, VerifyCode, RevokeDevice
│   ├── Commands/
│   ├── Interfaces/              → ISecretStore, ITokenRepository, IClock
│   └── Validators/              → FluentValidation for request DTOs
├── EToken.Domain/                → Entities, TOTP generation logic, no external deps
│   ├── Entities/                → TokenSecret, CustomerDevice
│   └── Services/                → TotpService (wraps Otp.NET)
├── EToken.Infrastructure/        → EF Core DbContext, repositories, KMS client
│   ├── Persistence/
│   └── Security/                 → KeyVaultSecretStore : ISecretStore
└── EToken.Tests/                 → Unit + integration tests
```

**Why this shape matters for a bank specifically:** the Domain project has zero dependencies on EF Core, Azure SDKs, or anything external — the TOTP algorithm and business rules (replay checks, drift tolerance) are pure C#, fully unit-testable without touching a database or a vault. This is what a security review will want to see isolated and independently verifiable.

## 3. Core domain logic

```
// EToken.Domain/Services/TotpService.cs
public interface ITotpService
{
    string GenerateCode(byte[] secret, DateTimeOffset time);
    bool Verify(byte[] secret, string submittedCode, DateTimeOffset serverTime, int driftSteps = 1);
}

public class TotpService : ITotpService
{
    private const int StepSeconds = 30;

    public string GenerateCode(byte[] secret, DateTimeOffset time)
    {
        var totp = new OtpNet.Totp(secret, step: StepSeconds, totpSize: 6);
        return totp.ComputeTotp(time.UtcDateTime);
    }

    public bool Verify(byte[] secret, string submittedCode, DateTimeOffset serverTime, int driftSteps = 1)
    {
        var totp = new OtpNet.Totp(secret, step: StepSeconds, totpSize: 6);
        return totp.VerifyTotp(
            serverTime.UtcDateTime,
            submittedCode,
```

```
                out _,
                new OtpNet.VerificationWindow(previous: driftSteps, future: driftSteps)
            );
        }
    }
```

Replay prevention and rate-limiting are **not** part of this service deliberately — they're orchestration concerns that belong in the Application layer, which has access to the repository (to check/update `last_accepted_bucket`) and to the attempt counter. Keeping `TotpService` stateless makes it trivial to unit test in isolation.

```
// EToken.Application/Commands/VerifyCodeHandler.cs
public class VerifyCodeHandler
{
    private readonly ITokenRepository _repo;
    private readonly ISecretStore _secretStore;
    private readonly ITotpService _totp;
    private readonly IClock _clock;

    public async Task<VerifyResult> Handle(VerifyCodeCommand cmd)
    {
        var lockStatus = await _repo.GetLockStatus(cmd.Cif);
        if (lockStatus.IsLocked)
            return VerifyResult.LockedOut(lockStatus.LockedUntil);

        var record = await _repo.GetActiveSecret(cmd.Cif, cmd.DeviceId);
        if (record is null)
            return VerifyResult.Invalid("no_active_token");

        var secret = await _secretStore.Decrypt(record.EncryptedSecret);
        var now = _clock.UtcNow;

        var currentBucket = now.ToUnixTimeSeconds() / 30;
        if (currentBucket <= record.LastAcceptedBucket)
            return VerifyResult.Invalid("replay");

        var valid = _totp.Verify(secret, cmd.Code, now);
        if (!valid)
        {
            await _repo.IncrementFailedAttempts(cmd.Cif);
            await _repo.LogAttempt(cmd.Cif, cmd.DeviceId, cmd.ActionType, success: false);
            return VerifyResult.Invalid("invalid_code");
        }

        await _repo.UpdateLastAcceptedBucket(record.Id, currentBucket);
        await _repo.ResetFailedAttempts(cmd.Cif);
        await _repo.LogAttempt(cmd.Cif, cmd.DeviceId, cmd.ActionType, success: true);
        return VerifyResult.Success();
    }
}
```

`IClock` is injected rather than calling `DateTimeOffset.UtcNow` directly — this is what makes the replay-prevention and drift-tolerance logic actually unit-testable (you can freeze time in a test and assert exact boundary behaviour, rather than trusting a live clock).

## 4. Database schema (EF Core / SQL)

```
CREATE TABLE customer_devices (
    device_id UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
    cif NVARCHAR(20) NOT NULL,
    device_model NVARCHAR(100),
    status NVARCHAR(20) NOT NULL DEFAULT 'active', -- active | revoked
    registered_at DATETIMEOFFSET NOT NULL DEFAULT SYSDATETIMEOFFSET(),
    revoked_at DATETIMEOFFSET NULL,
    INDEX ix_customer_devices_cif (cif)
);

CREATE TABLE token_secrets (
    id UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
    cif NVARCHAR(20) NOT NULL,
    device_id UNIQUEIDENTIFIER NOT NULL REFERENCES customer_devices(device_id),
    encrypted_secret VARBINARY(MAX) NOT NULL,   -- ciphertext from KMS/Key Vault
    last_accepted_bucket BIGINT NOT NULL DEFAULT 0,
    status NVARCHAR(20) NOT NULL DEFAULT 'active',
    created_at DATETIMEOFFSET NOT NULL DEFAULT SYSDATETIMEOFFSET(),
    INDEX ix_token_secrets_cif (cif)
);

CREATE TABLE verification_log (
    id BIGINT IDENTITY PRIMARY KEY,
    cif NVARCHAR(20) NOT NULL,
    device_id UNIQUEIDENTIFIER NOT NULL,
    action_type NVARCHAR(20) NOT NULL,   -- login | transaction | other
    result NVARCHAR(20) NOT NULL,        -- success | failed | locked_out
    ip_address NVARCHAR(45),
    created_at DATETIMEOFFSET NOT NULL DEFAULT SYSDATETIMEOFFSET(),
    INDEX ix_verification_log_cif_created (cif, created_at)
);

CREATE TABLE verification_attempts (
```

```
    cif NVARCHAR(20) PRIMARY KEY,
    failed_count INT NOT NULL DEFAULT 0,
    locked_until DATETIMEOFFSET NULL
);
```

EF Core entity configuration should mark `encrypted_secret` with `[Column(TypeName = "varbinary(max)")]` and ensure it's **excluded from any default logging/tracing output** (EF Core's sensitive data logging must stay off in every environment above local dev).

## 5. API surface (ASP.NET Core controllers)

```
[ApiController]
[Route("api/etoken")]
public class ETokenController : ControllerBase
{
    [HttpPost("enrol/init")]
    [Authorize] // requires an already-authenticated session
    public async Task<IActionResult> InitEnrolment(InitEnrolmentRequest req) { ... }

    [HttpPost("enrol/confirm")]
    [Authorize]
    public async Task<IActionResult> ConfirmEnrolment(ConfirmEnrolmentRequest req) { ... }

    [HttpPost("verify")]
    [Authorize]
    [EnableRateLimiting("etoken-verify")]
    public async Task<IActionResult> Verify(VerifyRequest req)
    {
        // CIF pulled from the authenticated principal, never trusted from the request body
        var cif = User.FindFirst("cif")!.Value;
        var result = await _mediator.Send(new VerifyCodeCommand(cif, req.DeviceId, req.Code, req.ActionType));
        return result.IsValid ? Ok(result) : BadRequest(result);
    }

    [HttpPost("revoke")]
    [Authorize]
    public async Task<IActionResult> Revoke(RevokeRequest req) { ... }
}
```

Rate limiting policy, registered in `Program.cs`:

```
builder.Services.AddRateLimiter(options =>
{
    options.AddFixedWindowLimiter("etoken-verify", opt =>
    {
        opt.PermitLimit = 5;
        opt.Window = TimeSpan.FromMinutes(15);
        opt.QueueLimit = 0;
    });
});
```

This is a defense-in-depth layer alongside the application-level `verification_attempts` lockout — the middleware limiter protects against raw request flooding, the DB-backed counter protects the specific CIF regardless of which IP or device the requests come from.

## 6. Security-critical implementation notes

- `ISecretStore` **implementation** in Infrastructure calls Key Vault/KMS for every encrypt/decrypt — never cache decrypted secrets in memory longer than the single verify operation, and never log them (add explicit `[JsonIgnore]` / excluded serialization on any DTO that could carry a raw secret).
- **Server time only.** Never accept a client-submitted timestamp anywhere in the verify path — `IClock.UtcNow` is the only source used in `VerifyCodeHandler`.
- **CIF from claims, not request body**, for any authenticated endpoint — the controller reads it off `User.FindFirst("cif")`, sourced from the JWT/session issued at login, not from anything the client sends in `VerifyRequest`.
- **Idempotent replay check** — `LastAcceptedBucket` update and the verify check must happen inside the same transaction/row-lock to avoid a race where two concurrent requests both read the old bucket before either writes the new one. Use `SELECT ... FOR UPDATE` (Postgres) or `UPDLOCK, ROWLOCK` hints (SQL Server) on the `token_secrets` row during verification.
- **Structured audit logging** — every `verification_log` write should go through a single logging path so it can't accidentally be skipped on an early return.

## 7. Testing strategy

- **Domain layer**: pure unit tests against `TotpService` — known secret + known time → known code (RFC 6238 test vectors), plus boundary tests for the drift window.
- **Application layer**: unit tests against `VerifyCodeHandler` with mocked `IClock`, `ITokenRepository`, `ISecretStore` — specifically test the replay-boundary case (`currentBucket == lastAcceptedBucket` must reject).
- **Infrastructure layer**: integration tests against a real (test) database and a KMS emulator/sandbox key, not mocks, to catch encryption round-trip issues.

- **API layer**: contract tests asserting `cif` is never accepted from the request body on `/verify`.

## 8. Deployment notes

- Run the API behind the bank's existing API gateway/WAF — this service should never be internet-facing on its own.
- Externalize all Key Vault/KMS configuration via environment-specific `appsettings.{Environment}.json` + managed identity (Azure) or IAM role (AWS) — no secrets or connection strings committed to source control.
- Health check endpoint (`/health`) should verify DB connectivity and Key Vault reachability, not just process liveness, since a KMS outage silently breaks every verification.